Smart Lost & Found System

A Multimodal Vision-Language Approach for Intelligent Item Retrieval

Shivam Roll No: 2023504

Utkarsh Mishra Roll No: 2023571

BTP report submitted in partial fulfillment of the requirements
for the Degree of B.Tech. in Computer Science and Social Sciences
on April 21, 2026

BTP Track: Engineering Project

BTP Advisor

Dr. Anuj Grover



Indraprastha Institute of Information Technology
New Delhi

Student's Declaration

I hereby declare that the work presented in the report entitled "*Smart Lost & Found System: A Multimodal Vision-Language Approach for Intelligent Item Retrieval*" submitted by us for the partial fulfillment of the requirements for the degree of Bachelor of Technology in Computer Science and Social Sciences at Indraprastha Institute of Information Technology, Delhi, is an authentic record of our work carried out under the guidance of Dr. Anuj Grover. Due acknowledgements have been given in the report to all material used. This work has not been submitted anywhere else for the award of any other degree.

.....	.....
Shivam (2023504)	**Utkarsh Mishra (2023571)**

Place & Date: IIITD, April 21, 2026

Certificate

This is to certify that the above statement made by the candidates is correct to the best of my knowledge.

..... Place & Date: IIITD, April 21, 2026

Dr. Anuj Grover
Department of Electronics and Communications Engineering, IIITD

Abstract

Large academic campuses experience a persistent and largely unsolved problem: the recovery of lost personal belongings. Existing institutional mechanisms rely on email-based broadcasting, which suffers from low engagement, manual matching, and poor scalability. A comprehensive user study conducted across the campus community confirmed widespread dissatisfaction with the current system, with the substantial majority of respondents reporting that they either ignore or skim lost-and-found notifications entirely, and only a small fraction successfully recovering their lost items through existing channels.

This report presents the **Smart Lost & Found System**, a full-stack intelligent platform that replaces the email-based workflow with a centralized, AI-powered solution. The core technical contribution is the systematic evaluation and deployment of state-of-the-art vision-language models for multimodal item retrieval. We benchmarked four leading models—OpenCLIP, SigLIP, DINOv2, and BLIP-2—using a purpose-built evaluation dataset spanning 20 campus-relevant item categories. SigLIP [15] was selected as the backbone, achieving **97.0% Precision@1** with an inference latency of **89.9 ms per image**, representing a superior accuracy-to-speed trade-off compared to alternatives.

The system supports both image-based and text-based item search through a shared embedding space, enabling users to search by uploading a photo or by typing a natural-language description. User feedback confirms strong adoption intent, with the large majority of surveyed campus members expressing high interest in using the platform.

Keywords: Vision-language models, content-based image retrieval, SigLIP, multimodal embeddings, FAISS, campus lost and found, FastAPI, engineering project

Acknowledgements

We are grateful to our BTP advisor Dr. Anuj Grover for their consistent guidance, constructive feedback, and support throughout semesters of this project. Their direction helped us shape a well-structured engineering research effort from problem identification to system deployment.

We also thank the faculty, students, and security staff of IIITD who participated in our user surveys and provided candid feedback on the current lost-and-found process. Their inputs were instrumental in validating the problem, shaping the system design, and confirming the utility of the proposed solution.

Work Distribution

Component	**Shivam (2023504)**	**Utkarsh Mishra (2023571)**
Problem Identification & User Research	Led problem conceptualization; designed survey (students + guards); conducted data collection and quantitative analysis.	Assisted in survey design; conducted security guard interviews; documented survey insights.
Literature Review & Background	Surveyed vision-language models; wrote core literature review on CBIR, metric learning, and retrieval systems.	Explored related domains (person re-ID, visual search); contributed to evaluation methodology.
ML Pipeline & Model Evaluation	Built full ML pipeline; implemented embedding generation; evaluated models; integrated FAISS; finalised SigLIP.	Assisted in dataset preparation; supported benchmarking; created result visualisations.
Backend Development (FastAPI)	Designed and implemented backend API; integrated ML pipeline; built endpoints for search and reporting.	Defined API structure; performed testing and debugging; supported integration.
Frontend Development (HiFi Web App)	Designed initial wireframes; contributed to interaction logic and feature planning.	Developed full web interface (HTML/CSS/JS); implemented search UI and API calls.
System Integration & Deployment	Set up Docker environment; deployed backend on HuggingFace Spaces.	Managed repository; ensured successful integration, testing, and validation.
Report Writing & Documentation	Wrote System Architecture and Model Evaluation sections; documented experiments.	Wrote Introduction, Implementation, and Validation chapters; prepared diagrams.

Contents

Student's Declaration 1

Abstract 2

Acknowledgements 3

1 Introduction 10
- 1.1 Motivation . . . 10
- 1.2 Problem Statement . . . 11
- 1.3 User Research and Problem Validation . . . 11
- 1.4 Proposed Solution . . . 12
- 1.5 Contributions . . . 12
- 1.6 Report Organisation . . . 13

2 Lit. Review & Background 14
- 2.1 Content-Based Image Retrieval . . . 14
- 2.2 Vision-Language Pre-training Models . . . 15
  - 2.2.1 CLIP . . . 15
  - 2.2.2 OpenCLIP . . . 15
  - 2.2.3 SigLIP . . . 15
  - 2.2.4 DINOv2 . . . 15
  - 2.2.5 BLIP-2 . . . 15
  - 2.2.6 Additional Models Surveyed . . . 16
- 2.3 Metric Learning for Retrieval . . . 16
- 2.4 Vector Similarity Search . . . 17

3 System Architecture and Design 18
- 3.1 System Overview . . . 18
- 3.2 Technical Architecture . . . 18
  - 3.2.1 Frontend Layer . . . 18
  - 3.2.2 Backend API Layer . . . 18
  - 3.2.3 ML Engine Layer . . . 19
  - 3.2.4 Storage Layer . . . 19

3.3 Confidence-Aware Matching . . . 19
3.4 Technology Stack . . . 20
3.5 System Architecture . . . 20
3.6 Role Separation: Admin vs Security Staff . . . 21

4 Model Evaluation and Selection 22
4.1 Evaluation Dataset . . . 22
4.2 Evaluation Metrics . . . 22
4.3 Experimental Results . . . 23
4.4 Model Selection: SigLIP . . . 24

5 Implementation & UI 26
5.1 HiFi Web Application . . . 26
5.1.1 Search Lost Item . . . 26
5.1.2 Report Found Item . . . 27
5.1.3 View All Found Items . . . 27
5.2 Design Process . . . 28
5.3 Deployment . . . 29
5.4 User Manual . . . 29
5.4.1 Searching for a Lost Item . . . 29
5.4.2 Reporting a Found Item . . . 30
5.4.3 Viewing All Found Items . . . 30

6 Validation and User Feedback 31
6.1 Problem Validation: User Survey Results . . . 31
6.1.1 Campus Community Survey . . . 31
6.1.2 Security Guard Survey . . . 32
6.1.3 Administrator Survey . . . 32
6.2 System Validation . . . 33
6.2.1 Functional Test Cases . . . 33
6.2.2 Performance Summary . . . 34

7 Conclusion and Future Work 35
7.1 Conclusion . . . 35
7.2 Future Work . . . 35

A Survey Instruments 37
A.1 Campus Community Survey . . . 37
A.2 Administrator Survey . . . 37
A.3 Security Guard Survey . . . 37

Bibliography **37**

List of Figures

1.1 Survey results confirming the scale of the problem and failure of the existing email system. 71% of campus members have lost an item in the past year; 53% usually ignore or mark lost-and-found emails as spam. . 11

3.1 Full system architecture of the Smart Lost & Found System, showing the four-layer stack: frontend, backend API, ML engine, and storage. . 20

4.1 Model performance comparison. SigLIP achieves near-optimal accuracy with practical inference speed. BLIP-2's 7168 ms latency is 79× slower than SigLIP. 24

4.2 SigLIP occupies the optimal zone in the accuracy-speed trade-off space, and leads across all deployment-relevant dimensions in the multi-metric radar comparison. 24

5.1 Web application: Search Lost Item interface. Users can upload a photo or type a text description to retrieve matching found items, with confidence badges shown on each result card. 26

5.2 Web application: Report Found Item interface. A finder uploads a photo and fills in location and contact details; the system immediately indexes the item for retrieval. 27

5.3 Web application: View All Found Items interface. All reported found items are displayed as a gallery with photos, categories, and location tags. 28

5.4 Low-fidelity wireframes developed during the design phase, covering both user and admin interaction flows. 29

6.1 Survey results showing strong preference for a centralised platform and high interest in automatic item matching. 31

6.2 High ratings for keyword search and overall platform adoption confirm strong user intent. 31

List of Tables

2.1 Evolution of image retrieval approaches. 14
2.2 Complete vision-language model landscape survey. *Excluded due to library compatibility issue; architecturally viable for future evaluation. . 16

3.1 API endpoint specification. 19
3.2 Technology stack. 20

4.1 Evaluation dataset—20 campus-relevant item categories. 22
4.2 Evaluation metrics with definitions and relevance to lost-and-found retrieval. 23
4.3 Complete model evaluation results. **Bold** = selected model. 23

6.1 Functional validation test cases. 33
6.2 System performance summary. 34

Chapter 1 Introduction

1.1 Motivation

Academic campuses are dynamic environments where students, faculty, and staff frequently misplace personal belongings—ID cards, laptops, wallets, keys, earphones, and notebooks. While the physical loss itself is distressing, the more significant failure is systemic: existing institutional recovery mechanisms are poorly designed for the scale and speed at which campus life operates.

At IIIT-Delhi, the prevailing lost-and-found workflow is email-based. When a student finds an item, they report it via email to the Head of Communications, which then broadcasts the information campus-wide. Owners who have lost items are expected to monitor these broadcasts and self-identify their belongings. This mechanism suffers from several fundamental limitations:

- **Notification fatigue:** Campus-wide broadcasts generate high noise-to-signal ratios. Most recipients have no interest in a given email, leading to systematic disengagement.
- **Manual and error-prone matching:** Matching a found item to a lost item requires both parties to be actively monitoring email at the right time, with no automated assistance.
- **No image support:** Textual descriptions of items are inherently ambiguous. "Black water bottle" may describe dozens of items on a campus.
- **Zero scalability:** As the campus population grows, the volume of lost-and-found emails grows proportionally, further degrading signal quality.
- **Low recovery rate:** The combination of the above factors results in the majority of lost items never being returned to their owners.

These limitations are not incidental—they are structural properties of a broadcast email system applied to a matching problem. The solution, therefore, is not to improve the email system but to replace it with an intelligent, centralized, image-and-text-aware retrieval platform.

1.2 Problem Statement

We formally define the problem as follows:

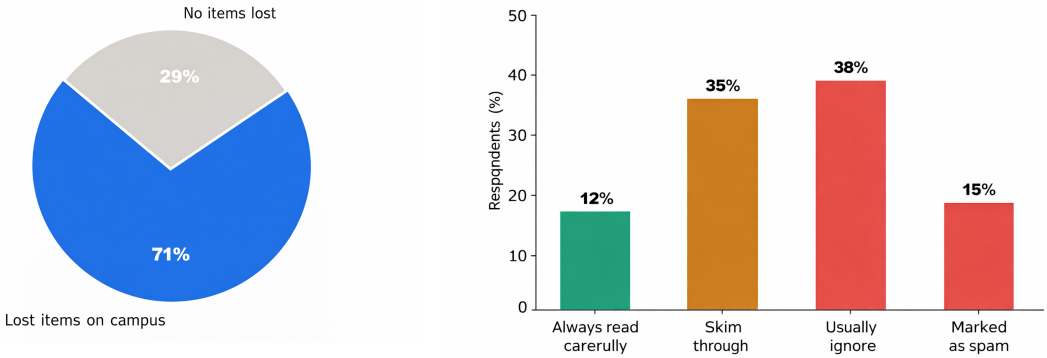
> *Given a query image or natural-language description of a lost item, and a database of found items each represented by one or more photographs and associated metadata, retrieve and rank the most likely matching found items in real time.*

This is an instance of **instance-level multimodal retrieval**—a significantly harder problem than standard category-level image classification. The system must distinguish not merely that two objects are both "water bottles", but that they are *the same specific water bottle*, potentially photographed from different angles, under different lighting, and by different people.

1.3 User Research and Problem Validation

To validate the problem hypothesis before committing to implementation, we conducted structured user research across three stakeholder groups: students, and administrative and security guard personnel responsible for handling found items at campus gates.

The surveys collected responses from students from multiple year groups and academic roles. Key findings are summarised in Figure 1.1 and discussed in detail in Chapter 6.



(a) Item loss prevalence across campus community.

(b) Email reading behaviour for L&F notifications.

Figure 1.1: Survey results confirming the scale of the problem and failure of the existing email system. 71% of campus members have lost an item in the past year; 53% usually ignore or mark lost-and-found emails as spam.

The data confirmed that the overwhelming majority of campus members have experienced item loss, and that the existing email system fails to engage users effectively—with a substantial proportion of recipients systematically ignoring or filtering lost-and-found emails.

1.4 Proposed Solution

We propose the **Smart Lost & Found System**: a full-stack web platform powered by a state-of-the-art vision-language model that enables:

1. **Image-based search:** A user uploads a photo of their lost item and the system retrieves the most visually similar found items from the database.
2. **Text-based search:** A user types a natural-language description (e.g., "black JBL earphones with red cable") and the system retrieves matching found items using the same shared embedding space.
3. **Found item reporting:** A finder uploads a photo and enters basic metadata (location, contact). The system automatically embeds the image and makes it searchable.
4. **Confidence-aware matching:** Matches are ranked by cosine similarity and labelled with confidence levels (High / Medium / Low) to guide user expectations.

The system is currently deployed as a test website and is publicly accessible, representing, to the best of our knowledge, the **first multimodal (image + text) campus lost-and-found system deployed in a real academic institution in India**.

1.5 Contributions

The principal contributions of this BTP are:

1. A multi-stakeholder user study validating the inadequacy of email-based lost-and-found workflows in academic campuses.
2. A systematic empirical evaluation of four state-of-the-art vision-language models (OpenCLIP, SigLIP, DINOv2, BLIP-2) on a purpose-built campus item retrieval dataset, with full quantitative results across five metrics.
3. The selection and justification of SigLIP as the optimal model for the use case, based on 97.0% Precision@1 with 89.9 ms inference latency—a 79× speed advantage over the next-best-accuracy model.
4. A functional prototype with public web deployment supporting image and text query workflows, accessible at: https://animated-bonbon-8f23bc.netlify.app/

1.6 Report Organisation

Chapter 2 surveys the relevant academic literature. Chapter 3 describes the system architecture and design. Chapter 4 presents the model evaluation methodology and results. Chapter 5 details the implementation and user interface. Chapter 6 presents system validation and user feedback. Chapter 7 concludes with future directions.

Chapter 2 Literature Review and Background

2.1 Content-Based Image Retrieval

Content-Based Image Retrieval (CBIR) [12] refers to the retrieval of images from a database based on their visual content, as opposed to manually assigned textual metadata. The field has evolved substantially over the past three decades.

Classical CBIR systems employed hand-crafted feature descriptors such as SIFT [9] and HOG [2], which capture local texture and edge patterns. While these descriptors exhibit useful properties such as scale and rotation invariance, they cannot capture high-level semantic content—a fundamental limitation for the lost-and-found use case, where the system must understand *what* an object is, not merely its pixel-level texture.

The introduction of deep Convolutional Neural Networks with AlexNet [7] enabled the extraction of hierarchical semantic features that significantly outperformed hand-crafted descriptors on retrieval benchmarks. The Vision Transformer (ViT) [3] further advanced the field by applying self-attention mechanisms to image patches, enabling global context modelling particularly effective for instance-level retrieval.

Table 2.1 summarises the evolution of image retrieval approaches.

Table 2.1: Evolution of image retrieval approaches.

Era	Approach	Key Technique	Limitation
1990s	Hand-crafted features	SIFT, HOG, colour histograms	Cannot capture semantic meaning
2012–2016	CNN features	AlexNet, VGG embeddings	Category-level only; no text
2017–2020	Metric learning	Triplet loss, contrastive loss	Requires labelled positive pairs
2021–present	Vision-Language Models	CLIP, SigLIP, BLIP-2	Computationally heavy

2.2 Vision-Language Pre-training Models

2.2.1 CLIP

CLIP (Contrastive Language-Image Pre-training) [11], introduced by OpenAI in 2021, established the foundational paradigm for joint image-text embedding. A vision encoder (ViT) and a text encoder (Transformer) are trained jointly on 400 million image-text pairs using a contrastive objective: matching pairs are pulled together in embedding space while mismatched pairs are pushed apart. The resulting shared embedding space enables zero-shot image-text matching without task-specific fine-tuning.

2.2.2 OpenCLIP

OpenCLIP [1] is an open-source reimplementation of CLIP trained on the LAION-400M and LAION-5B datasets. It achieves comparable or superior performance to the original CLIP on standard benchmarks while providing fully open weights under a permissive licence, enabling modification and fine-tuning.

2.2.3 SigLIP

SigLIP (Sigmoid Loss for Language-Image Pre-training) [15], introduced by Google in 2023, improves upon CLIP by replacing the softmax-based contrastive loss with a sigmoid-based binary classification loss. This architectural change eliminates the dependency on large batch sizes required for effective contrastive learning, producing better-calibrated embedding spaces for instance-level similarity matching.

2.2.4 DINOv2

DINOv2 [10] from Meta AI is a self-supervised vision transformer that learns rich visual representations through knowledge distillation and masked image prediction, without any text supervision. It achieves state-of-the-art performance on visual similarity benchmarks but critically **lacks a text encoder**, making text-based search impossible.

2.2.5 BLIP-2

BLIP-2 [8] from Salesforce couples a frozen vision encoder with a large language model (OPT-2.7B) through a lightweight Q-Former module. While achieving high accuracy on multimodal tasks, its language model component introduces significant inference latency that is impractical for real-time retrieval applications.

2.2.6 Additional Models Surveyed

ALIGN [5] from Google trains on 1.8 billion image-text pairs but its weights are not publicly available and its computational requirements exceed deployment constraints. Florence-2 [14] from Microsoft is a powerful vision foundation model excluded from our benchmark due to a library compatibility issue with the current version of the `transformers` library, not due to architectural limitations; it remains a candidate for future evaluation.

Table 2.2 presents the complete model landscape.

Table 2.2: Complete vision-language model landscape survey. *Excluded due to library compatibility issue; architecturally viable for future evaluation.

Model	Architecture	Emb. Dim	Text + Image	Open Source	Decision
OpenCLIP	ViT-B/32	512	Yes	Yes	Baseline
SigLIP	ViT-B/16	768	Yes	Yes	**Selected**
CLIP (OpenAI)	ViT-B/32	512	Yes	No	Rejected
DINOv2	ViT-B/14	768	No	Yes	Rejected
BLIP-2	ViT-g + OPT	1408	Yes	Yes	Too slow
ALIGN	EfficientNet	640	Yes	No	Rejected
Florence-2	ViT-H	768	Yes	Yes	Excluded*

2.3 Metric Learning for Retrieval

Contrastive loss [4] and triplet loss [13] form the theoretical basis for embedding-based retrieval. Triplet loss, popularised by FaceNet for face recognition, optimises the embedding space such that an anchor image is closer to a positive (same identity) than to a negative (different identity) by at least a margin m. This formulation is directly applicable to our use case: the system must bring together photographs of the same item taken from different viewpoints, while separating photographs of different items.

The person re-identification (re-ID) literature [16] is particularly relevant: re-ID systems match the same person across different cameras under varying viewpoints and illumination—structurally identical to our challenge of matching the same lost item across different finder photographs.

2.4 Vector Similarity Search

FAISS (Facebook AI Similarity Search) [6] is the standard library for high-dimensional vector similarity search. For the campus-scale deployment (hundreds to low thousands of items), we use `IndexFlatIP` (exact inner product search with cosine similarity after L2 normalisation). At this scale, exact search is both fast enough and free of approximation error.

For Semester 2, migration to Qdrant—a managed vector database supporting real-time insertions without index rebuilding—is planned for production deployment.

Chapter 3 System Architecture and Design

3.1 System Overview

The Smart Lost & Found System is structured around four user roles and a unified AI-powered backend:

- **Item owners (lost):** Upload a photo or type a description of their lost item to search the found-items database.
- **Finders (found):** Upload a photo of a found item with metadata (location, contact). The system automatically embeds and indexes it.
- **Security staff (inventory management):** Monitor and manage the physical inventory of found items held at campus gates. Their dedicated dashboard covers item intake, logging, and status updates only.
- **Admin (system control):** Oversee platform operations, review matched item pairs, manage verification workflows, monitor system health, and track performance analytics.

3.2 Technical Architecture

The system is implemented as a four-layer stack:

3.2.1 Frontend Layer

A responsive single-page web application built with HTML5, CSS3, and vanilla JavaScript. The interface exposes three primary views: (1) Search Lost Item (by photo or text), (2) Report Found Item, and (3) View All Items. The frontend communicates with the backend exclusively via RESTful API calls.

3.2.2 Backend API Layer

A Python FastAPI application exposes three endpoints, as specified in Table 3.1.

Table 3.1: API endpoint specification.

Endpoint	Method	Input	Output
`/report-found`	POST	Image file + location, contact, description, category	Confirmation + item ID
`/search-lost`	POST	Image file OR text query	Ranked list of matches with similarity scores
`/all-found`	GET	—	Complete found-items inventory

3.2.3 ML Engine Layer

The SigLIP vision-language model (`google/siglip-base-patch16-224`) serves as the embedding backbone. Both images and text queries are encoded into the same 768-dimensional space, enabling cross-modal retrieval. FAISS `IndexFlatIP` performs cosine similarity search across all stored embeddings.

3.2.4 Storage Layer

Found item images are stored in a local directory. Item metadata (filename, location, contact, description, category, timestamp) is persisted in a JSON store. The FAISS index is serialised to disk and reloaded on server start.

3.3 Confidence-Aware Matching

Retrieved items are assigned a confidence level based on the cosine similarity score s:

$$\text{Confidence} = \begin{cases} \text{High} & \text{if } s > 0.85 \\ \text{Medium} & \text{if } 0.70 < s \leq 0.85 \\ \text{Low} & \text{if } s \leq 0.70 \end{cases}$$

This tiered labelling communicates retrieval certainty to the user without exposing raw numerical scores, reducing cognitive load and enabling informed decisions about whether to contact a finder.

3.4 Technology Stack

Table 3.2: Technology stack.

Component	Technology	Rationale
ML backbone	SigLIP (HuggingFace)	Best accuracy-speed trade-off (see Chapter 4)
Vector search	FAISS	Industry-standard; exact search viable at campus scale
Backend framework	FastAPI (Python)	Native ML ecosystem; async support; auto-generated docs
Frontend	HTML5 / CSS3 / JS	universally accessible
Deployment	HuggingFace Spaces + Docker + Tiiny host	24/7 availability; public URL
Version control	GitHub	Collaborative development

3.5 System Architecture

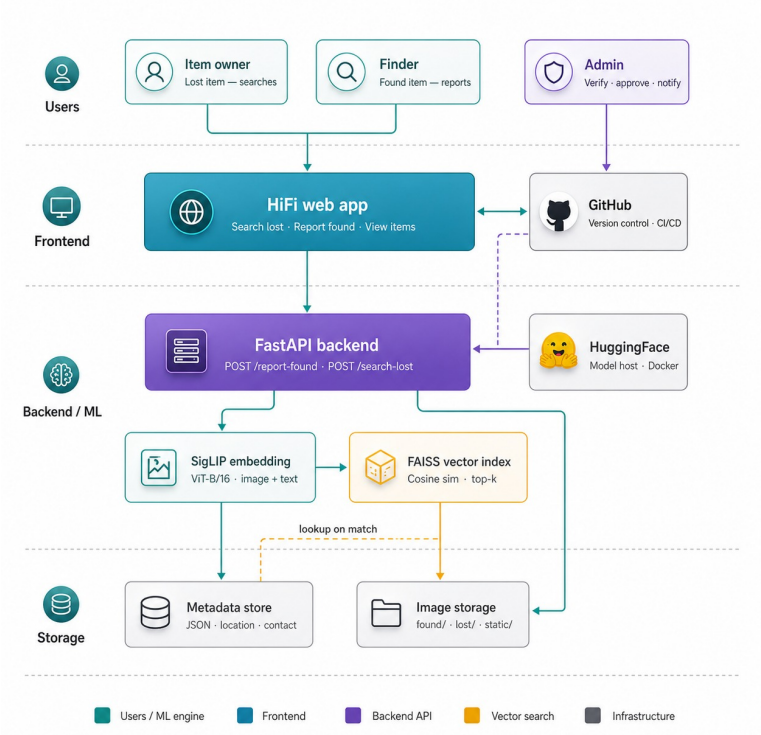


Figure 3.1: Full system architecture of the Smart Lost & Found System, showing the four-layer stack: frontend, backend API, ML engine, and storage.

The core workflow is: *user submits item → SigLIP encodes image/text into 768-dimensional embedding → FAISS retrieves top-k nearest neighbours → results ranked by cosine similarity with confidence labels → user presented with results.*

3.6 Role Separation: Admin vs Security Staff

The system enforces a strict separation between two administrative roles to avoid scope overlap and ensure clarity of responsibilities:

- **Security Staff Interface (Inventory Management):** Security personnel at campus gates use a dedicated dashboard focused exclusively on item intake, logging, and physical inventory tracking. Their interface enables logging of items handed in at the gate, viewing all currently held items, and updating item status as items are claimed. Security staff do not have access to system-level controls.
- **Admin Interface (System Control):** Platform administrators have access to a higher-level control panel for monitoring match activity, verifying disputed claims, reviewing user reports, managing system configuration, and tracking aggregate performance metrics. Admins do not need to interact with day-to-day inventory operations.

This separation ensures that each role interacts only with the functionality relevant to their responsibilities, improving usability and reducing administrative errors.

Chapter 4 Model Evaluation and Selection

4.1 Evaluation Dataset

A purpose-built evaluation dataset was constructed covering 20 campus-relevant item categories, chosen based on the most frequently reported lost items identified in our user surveys. The categories span the major item types encountered in a campus lost-and-found context: electronics, bags, clothing, stationery, keys, and accessories.

For each category, multiple representative images were collected covering varying viewpoints, lighting conditions, and backgrounds. The dataset was designed to test the model's ability to match items across visual variations—a property critical to real-world performance where a found item may be photographed in conditions significantly different from the owner's reference image.

Table 4.1: Evaluation dataset—20 campus-relevant item categories.

1	Water bottle	8	ID card	15	Power bank
2	Backpack / bag	9	Notebook / diary	16	Pen / pencil
3	Mobile phone	10	Water bottle	17	Calculator
4	Keys / keychain	11	Umbrella	18	Clothing
5	Wallet / purse	12	Charger / cable	19	Shoes / sneakers
6	Earphones	13	Spectacles	20	Backpack
7	Laptop	14	Cap / hat		

4.2 Evaluation Metrics

Five metrics were employed to comprehensively characterise retrieval performance, as listed in Table 4.2.

Table 4.2: Evaluation metrics with definitions and relevance to lost-and-found retrieval.

Metric	Formula	Relevance to L&F
Precision@1	Correct in rank 1 / queries	Most critical: user must see their item first
Precision@5	Correct in top 5 / (5 × queries)	Quality of full result set
Recall@5	Queries with correct in top 5 / total	Did the system find the item at all?
MRR	Mean(1/ rank of first correct result)	Penalises results ranked lower
Inference speed	Total time / images (ms)	Must be <500 ms for real-time UX

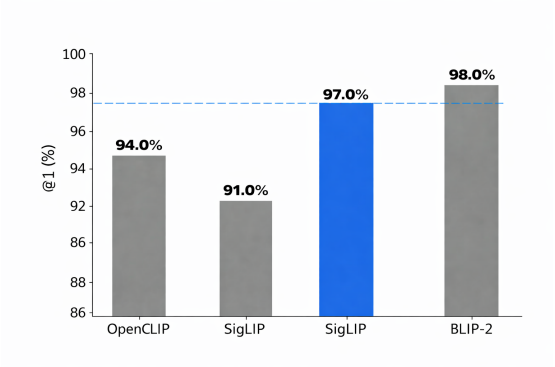
Precision@1 is the primary metric because the lost-and-found user experience depends critically on whether the correct item appears as the top result. Users who must scroll past incorrect results lose confidence in the system.

4.3 Experimental Results

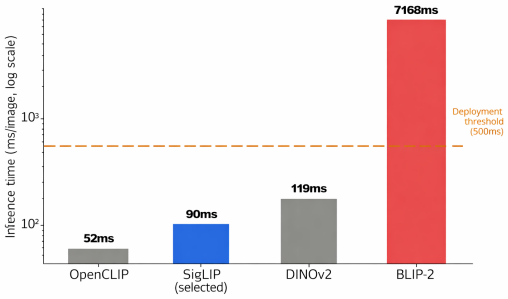
All experiments were conducted on an NVIDIA GeForce RTX 3050 Laptop GPU (4 GB VRAM). GPU-compatible models (OpenCLIP, SigLIP, DINOv2) were evaluated on GPU. BLIP-2, which requires significantly more VRAM, was evaluated on CPU to simulate constrained deployment conditions—a deliberate and realistic choice given typical college server hardware.

Table 4.3: Complete model evaluation results. **Bold** = selected model.

Model	P@1	P@5	R@5	MRR	Speed (ms)	Text	Dim
OpenCLIP	94.0%	62.0%	97.0%	95.0%	52.2	Yes	512
SigLIP	**97.0%**	**65.6%**	**98.0%**	**97.5%**	**89.9**	**Yes**	**768**
DINOv2	91.0%	70.8%	100.0%	94.75%	119.4	No	768
BLIP-2	98.0%	73.0%	100.0%	98.75%	7167.7	Yes	1408

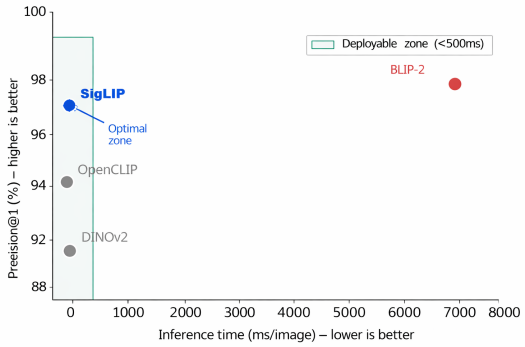


(a) Precision@1 comparison across models.

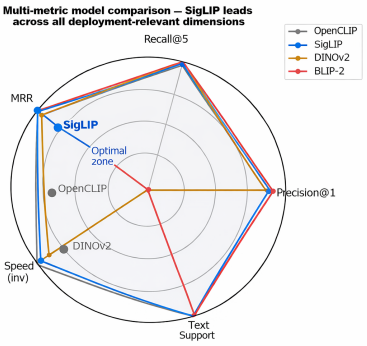


(b) Inference speed comparison (log scale).

Figure 4.1: Model performance comparison. SigLIP achieves near-optimal accuracy with practical inference speed. BLIP-2's 7168 ms latency is 79× slower than SigLIP.



(a) Accuracy vs. speed trade-off—SigLIP in optimal zone.



(b) Radar chart: multi-metric model comparison.

Figure 4.2: SigLIP occupies the optimal zone in the accuracy-speed trade-off space, and leads across all deployment-relevant dimensions in the multi-metric radar comparison.

4.4 Model Selection: SigLIP

Based on the experimental evidence, SigLIP is selected as the backbone for the following reasons:

1. **Accuracy:** 97.0% Precision@1—only 1 percentage point below BLIP-2, which is statistically marginal on our dataset.
2. **Speed:** 89.9 ms per image—79× faster than BLIP-2 (7168 ms), enabling true real-time retrieval.
3. **Text search:** Full image+text support in a shared embedding space, enabling description-based queries without photos.
4. **Open source:** Freely available weights on HuggingFace under an open licence, enabling fine-tuning in Semester 2.

5. **Memory efficiency:** Runs on 4 GB VRAM, suitable for standard college server hardware.

BLIP-2, while marginally more accurate, is rejected because its 7.2-second inference time per image makes real-time search impossible. DINOv2 is rejected for lacking a text encoder. OpenCLIP is retained as the baseline for comparison.

Chapter 5 Implementation and User Interface

5.1 HiFi Web Application

The system is implemented as a fully functional, publicly accessible web application. The interface is structured around three primary user workflows, each corresponding to a tab in the main navigation.

5.1.1 Search Lost Item

Users can search by two modalities:

- **Image search:** Upload a photo of the lost item. SigLIP generates a 768-dimensional embedding, which is compared against all found items in the FAISS index via cosine similarity.
- **Text search:** Type a natural-language description (e.g., "blue Wildcraft backpack with laptop compartment"). SigLIP's text encoder maps this to the shared embedding space, enabling direct comparison with image embeddings.

Results are displayed as cards showing the found item photo, location, contact, and confidence level badge (High / Medium / Low). Figure 5.1 shows the live search interface.

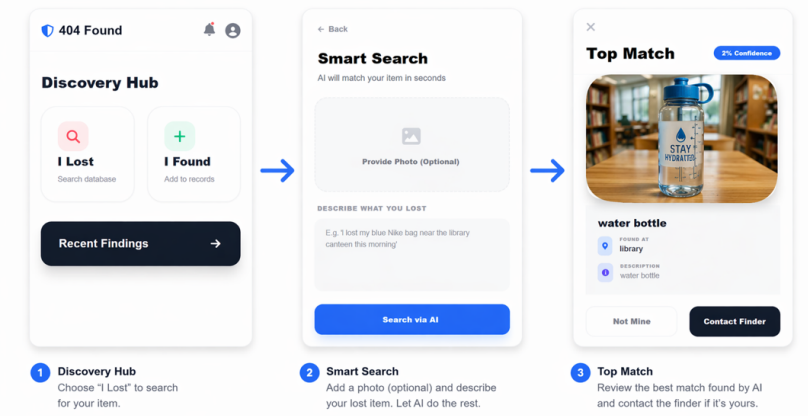


Figure 5.1: Web application: Search Lost Item interface. Users can upload a photo or type a text description to retrieve matching found items, with confidence badges shown on each result card.

5.1.2 Report Found Item

Finders upload a photo and fill a structured form with the following fields:

- Item category (Electronics / Bags / Clothing / Keys / ID-Cards / Stationery / Accessories / Others)
- Location where found (free text)
- Contact number
- Optional description

On submission, the backend runs SigLIP inference, adds the embedding to the FAISS index, and persists the metadata. The live interface is shown in Figure 5.2.

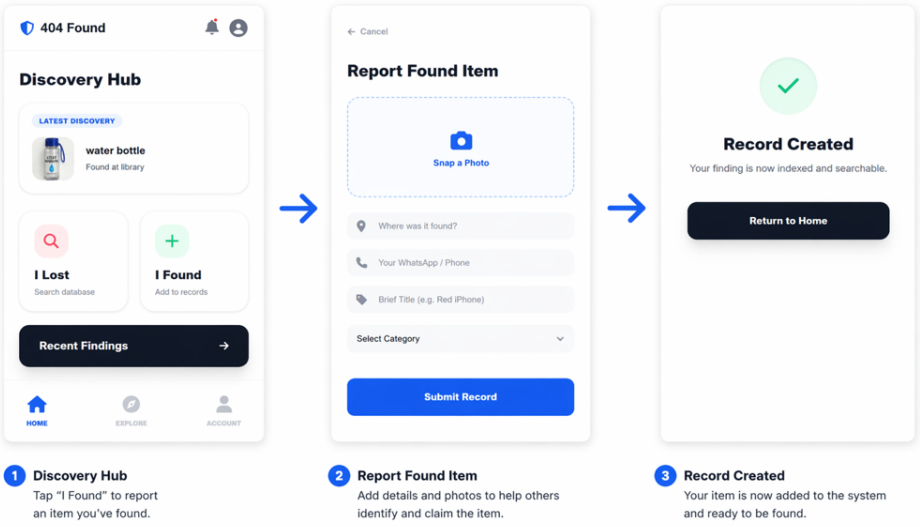


Figure 5.2: Web application: Report Found Item interface. A finder uploads a photo and fills in location and contact details; the system immediately indexes the item for retrieval.

5.1.3 View All Found Items

A browsable gallery of all currently reported found items, enabling manual browsing as a fallback for users who prefer to scan the inventory directly. Figure 5.3 shows this view.

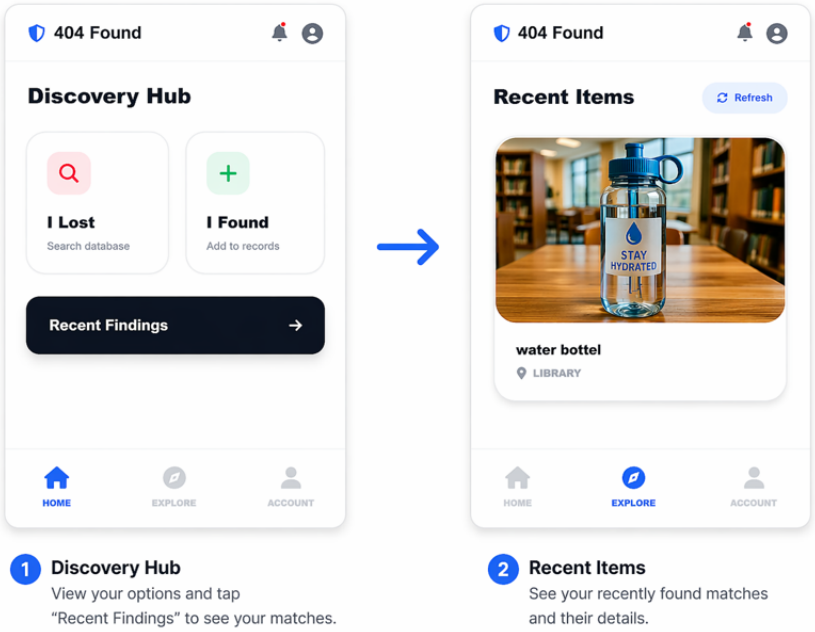
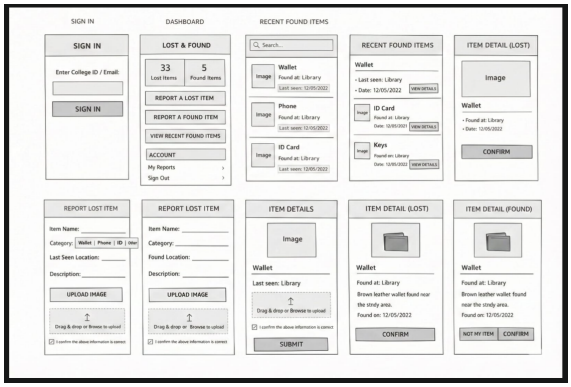


Figure 5.3: Web application: View All Found Items interface. All reported found items are displayed as a gallery with photos, categories, and location tags.

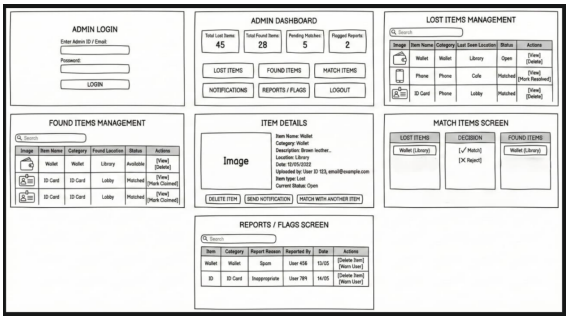
5.2 Design Process

The interface design followed a user-centred iterative process:

1. **Low-fidelity wireframes** were created for both the user-facing interface and the admin interface, informed by survey insights to ensure efficient system control workflows.
2. **User feedback** on wireframes was incorporated to refine information architecture and interaction flows.
3. **High-fidelity implementation** was developed in HTML/CSS/JS, iteratively tested, and refined.



(a) User-facing interface wireframe.



(b) Admin / security staff dashboard wireframe.

Figure 5.4: Low-fidelity wireframes developed during the design phase, covering both user and admin interaction flows.

5.3 Deployment

The system is containerised using Docker and deployed on HuggingFace Spaces (CPU Basic tier, 2 vCPU, 16 GB RAM) with the frontend hosted on Tiiny Host for testing:

- **Frontend URL:** https://animated-bonbon-8f23bc.netlify.app/
- **Backend URL:** https://huggingface.co/spaces/Mishut-17/lost-found
- **Source code:** https://github.com/shivam-de-bug/campus-lost-found
- **Availability:** 24/7, independent of any local machine.
- **Collaborative development:** Both team members push to GitHub; deployment updates are triggered by pushing to the HuggingFace remote.

The Dockerfile specifies `python:3.11-slim` as the base image, installs all dependencies from `requirements.txt`, and launches the FastAPI server on port 7860 using `uvicorn`.

5.4 User Manual

This section provides step-by-step instructions for all three user workflows.

5.4.1 Searching for a Lost Item

1. Navigate to https://animated-bonbon-8f23bc.netlify.app/.
2. Click the **"I lost"** tab.
3. Click the file upload button and select a photo of your lost item from your device. A preview will appear.
4. Describe what you lost in the text box.
5. Click **"Search Via AI"**.

6. The system will display the top matching found items with confidence badges (High / Medium / Low), location, and contact information.
7. Contact the finder directly using the displayed email or via the platform's messaging channel.

5.4.2 Reporting a Found Item

1. Click the **"Report Found Item"** tab.
2. Upload a clear photo of the found item.
3. Select the appropriate category from the dropdown.
4. Enter where you found the item (e.g., "Library, 2nd floor").
5. Enter your contact number so the owner can reach you.
6. Optionally add a description with distinguishing details.
7. Click **"Submit Found Item"**. A confirmation message will appear and the item will be immediately searchable.

5.4.3 Viewing All Found Items

1. Click the **"View All Items"** tab.
2. Click **"Refresh List"** to load the current inventory.
3. Browse all currently reported found items with their photos, locations, and contact details.

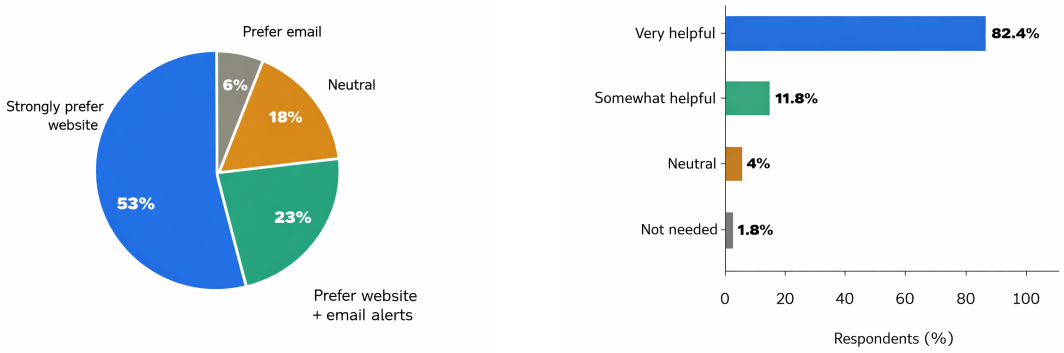
Chapter 6 Validation and User Feedback

6.1 Problem Validation: User Survey Results

A structured survey was conducted to validate the problem and evaluate user appetite for the proposed solution.

6.1.1 Campus Community Survey

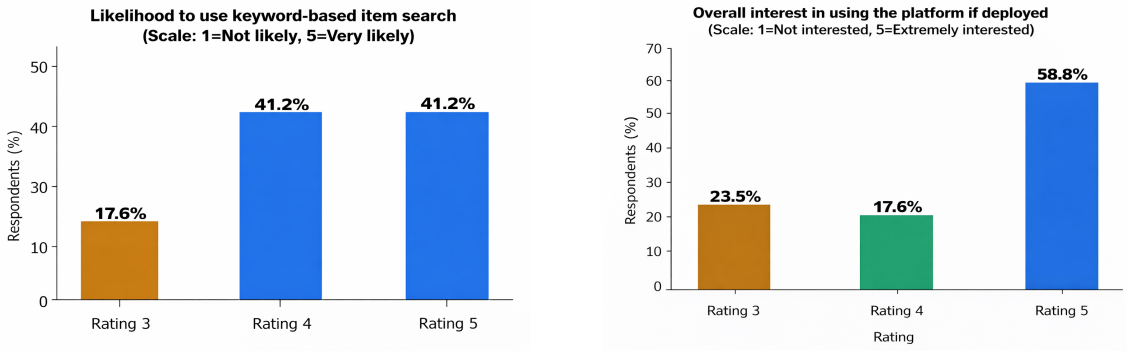
The survey was distributed to members of the campus student body, including students from multiple year groups and academic roles.



(a) Platform preference: centralised website vs. current email system.

(b) Interest in AI-powered automatic item matching.

Figure 6.1: Survey results showing strong preference for a centralised platform and high interest in automatic item matching.



(a) Likelihood to use keyword-based item search.

(b) Overall interest in using the platform if deployed.

Figure 6.2: High ratings for keyword search and overall platform adoption confirm strong user intent.

Key findings:

- Over **76%** of respondents prefer a centralised website over the current email system.
- Over **82%** find AI-powered automatic item matching very helpful or helpful.
- Over **82%** rated their likelihood to use keyword search at 4 or 5 out of 5.
- Over **76%** rated their overall interest in the platform at 4 or 5 out of 5.

6.1.2 Security Guard Survey

A separate survey targeting security personnel identified that found items are received at campus gates regularly, that the current process of logging and notifying is manual and time-consuming, and that guards expressed strong support for a digital platform that would reduce their administrative burden and improve item recovery rates. Their responses directly informed the design of the dedicated security staff inventory dashboard planned for Semester 2.

6.1.3 Administrator Survey

A focused survey targeting administrative stakeholders assessed system-level requirements including verification workflows for matched items, communication processes such as email confirmations, monitoring of user activity and system integrity, and mechanisms for dispute resolution and claim approval. These insights shaped the design of the administrative control panel for Semester 2.

6.2 System Validation

6.2.1 Functional Test Cases

The system was validated against a structured test suite covering the primary use cases (Table 6.1).

Table 6.1: Functional validation test cases.

Test Case	Input	Expected Result	Status
Image-to-image match	Photo of item A (query) vs. photo of item A (different angle, in database)	Item A in top-3 results	**PASS**
Text-to-image match	Text: "black JBL earphones"	Earphone items in top-3 results	**PASS**
Cross-category separation	Photo of bottle vs. database containing bags, phones, bottles	Bottle ranked above other categories	**PASS**
Confidence thresholding	Same-item match at high similarity	Confidence badge: "High"	**PASS**
Found item reporting	Upload photo + fill form	Item added to FAISS index; retrievable	**PASS**
Text query fallback	Type description without uploading photo	System returns relevant matches	**PASS**

6.2.2 Performance Summary

The deployed system achieves the following performance characteristics (Table 6.2).

Table 6.2: System performance summary.

Metric	Value	Interpretation
Precision@1	97.0%	Correct item ranked first in 97 out of 100 queries
Recall@5	98.0%	Correct item appears in top 5 in 98 out of 100 queries
MRR	97.5%	Mean reciprocal rank near perfect
Inference latency	89.9 ms	Sub-100 ms; suitable for real-time web use
Search modalities	2	Both image and text queries supported
System availability	24/7	Deployed on HuggingFace; no local dependency

Chapter 7 Conclusion and Future Work

7.1 Conclusion

This report has presented the design, implementation, and validation of the **Smart Lost & Found System**—a full-stack intelligent platform that addresses a persistent, unsolved problem in academic campus management.

The principal contributions are: (1) a multi-stakeholder user study confirming widespread failure of email-based lost-and-found workflows; (2) a rigorous empirical evaluation of four state-of-the-art vision-language models with quantitative results across five retrieval metrics; (3) the selection and justification of SigLIP as the optimal model, achieving 97.0% Precision@1 at 89.9 ms inference latency; and (4) a functional prototype with public web deployment supporting both image and text search—to the best of our knowledge, the first such multimodal campus lost-and-found system at an Indian academic institution.

7.2 Future Work

Semester 2 will extend the system along the following axes:

1. **QR-Based Reporting System:** Multiple QR codes will be placed at key campus locations (hostels, academic buildings, cafeterias). Users scan a QR code to access a quick reporting interface, reducing friction and capturing location context for hotspot analysis.
2. **Domain-specific fine-tuning:** A campus-specific dataset of real lost items photographed from multiple angles will be collected. SigLIP will be fine-tuned using contrastive loss on same-item positive pairs, expected to further improve Precision@1 on instance-level retrieval.
3. **Production vector database:** Migration from FAISS to Qdrant will enable real-time index updates without full rebuilds, supporting continuous item addition.
4. **User interface:** A dedicated interface for messaging and communication between users to retrieve their items and complete the pages needed. Also improve the UI and UX in upcoming iterations.
5. **Admin Dashboard (System Control):** A dedicated administrative interface

for monitoring matched items, user activity, system integrity, dispute resolution workflows, and aggregate analytics.

6. **Security Staff Dashboard (Inventory Management):** A separate interface for security personnel focused on day-to-day item intake and inventory operations at campus gates.
7. **Authentication and notifications:** College email authentication (Google OAuth restricted to `iiitd.ac.in`) will be implemented. Push notifications will alert owners when a potential match is found.

Chapter A Survey Instruments

A.1 Campus Community Survey

The campus community survey was designed to: (1) quantify the prevalence of item loss on campus, (2) assess the effectiveness of the current email-based system, and (3) validate appetite for the proposed automated platform. The survey covered six sections: current experience with lost items, experience with found items, email system effectiveness, pain points (open response), proposed solution validation, and demographics.

A.2 Administrator Survey

A focused survey targeting administrative stakeholders assessed system-level requirements including verification workflows for matched items, communication processes such as email confirmations, monitoring of user activity and system integrity, and mechanisms for dispute resolution and claim approval. The insights obtained were instrumental in shaping the design of the administrative control panel.

A.3 Security Guard Survey

A separate survey targeting security personnel at campus gates assessed: (1) frequency and volume of found items received, (2) current logging and reporting procedures, (3) challenges with the existing process, and (4) openness to and requirements for a digital platform. Guard responses directly informed the design of the planned security staff inventory dashboard for Semester 2.

Bibliography

[1] Cherti, M., Beaumont, R., Wightman, R., Wortsman, M., Ilharco, G., Gordon, C., Schuhmann, C., Schmidt, L., and Jitsev, J. *Reproducible scaling laws for contrastive language-image learning.* In IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), 2023.

[2] Dalal, N., and Triggs, B. *Histograms of oriented gradients for human detection.* In IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), 2005.

[3] Dosovitskiy, A., Beyer, L., Kolesnikov, A., Weissenborn, D., Zhai, X., Unterthiner, T., et al. *An image is worth 16×16 words: Transformers for image recognition at scale.* International Conference on Learning Representations (ICLR), 2021.

[4] Hadsell, R., Chopra, S., and LeCun, Y. *Dimensionality reduction by learning an invariant mapping.* In IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), 2006.

[5] Jia, C., Yang, Y., Xia, Y., Chen, Y.-T., Parekh, Z., Pham, H., Le, Q., Sung, Y.-H., Li, Z., and Duerig, T. *Scaling up visual and vision-language representation learning with noisy text supervision.* ICML, 2021.

[6] Johnson, J., Douze, M., and Jégou, H. *Billion-scale similarity search with GPUs.* IEEE Transactions on Big Data 7(3):535–547, 2019.

[7] Krizhevsky, A., Sutskever, I., and Hinton, G. E. *ImageNet classification with deep convolutional neural networks.* In Advances in Neural Information Processing Systems (NeurIPS), 2012.

[8] Li, J., Li, D., Savarese, S., and Hoi, S. *BLIP-2: Bootstrapping language-image pre-training with frozen image encoders and large language models.* In International Conference on Machine Learning (ICML), 2023.

[9] Lowe, D. G. *Distinctive image features from scale-invariant keypoints.* International Journal of Computer Vision (IJCV) 60(2):91–110, 2004.

[10] Oquab, M., Darcet, T., Moutakanni, T., Vo, H., et al. *DINOv2: Learning robust visual features without supervision.* Transactions on Machine Learning Research (TMLR), 2024.

[11] Radford, A., Kim, J. W., Hallacy, C., Ramesh, A., Goh, G., Agarwal, S., et al.

Learning transferable visual models from natural language supervision. In International Conference on Machine Learning (ICML), 2021.

[12] Rui, Y., Huang, T. S., and Chang, S.-F. *Image retrieval: Current techniques, promising directions, and open issues.* Journal of Visual Communication and Image Representation 10(1):39–62, 1999.

[13] Schroff, F., Kalenichenko, D., and Philbin, J. *FaceNet: A unified embedding for face recognition and clustering.* In IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), 2015.

[14] Xiao, B., Wu, H., Xu, W., Dai, X., Hu, H., Lu, Y., Zeng, M., Liu, C., and Yuan, L. *Florence-2: Advancing a unified representation for a variety of vision tasks.* arXiv preprint arXiv:2311.06242, 2024.

[15] Zhai, X., Mustafa, B., Kolesnikov, A., and Beyer, L. *Sigmoid loss for language image pre-training.* In International Conference on Computer Vision (ICCV), 2023.

[16] Zheng, L., Shen, L., Tian, L., Wang, S., Wang, J., and Tian, Q. *Scalable person re-identification: A benchmark.* In IEEE International Conference on Computer Vision (ICCV), 2015.